

HEdClass Web App Summary Report

1. Introduction

HEdClass is a Web App designed to offer a classification platform for Higher Education institutes. The system automates the manual process of classifying undergraduate students to determine final award outcomes. The current manual approach relies on spreadsheet software that increases the risk of human error, provides little to no auditability, and is not scalable to the growing number of students an institute will enrol. HedClass solves this problem by providing a deterministic, fair, and regulatory-compliant automated system. The application is structured around a traditional three-tier architecture. The back-end application logic is handled by *Node.js* and *Express.js*, while the frontend presentation layer uses *EJS* for dynamic page rendering and *Bootstrap* for a responsive user interface. All persistent data, including student records, module marks, and classification rules, is managed through a MySQL database. Development adhered strictly to the MoSCoW prioritisation method. As will be detailed further in this report, every single requirement outlined in the project brief has been successfully implemented.

2. Access Details

<https://gitlab.eeecs.gub.ac.uk/40083161/hedclass>

Access has been granted to the module tutor to allow for full code review and authorship validation. The application is configured to run on **PORT 3000** and the API is configured on **PORT 4000**. The database was hosted on a MySQL server with the database named according to my student ID (40083161). For the testing of the app I generated user accounts outlined in the table below.

Role	Email	Password
Institutional Administrator	gandalf@gub.ac.uk	yoshlIntpass!1
Classification Officer	holmes@gub.ac.uk	elementary2\$
Classification Officer	kenobi@gub.ac.uk	helloth3re

Detailed installation steps, including dependency installation via NPM and database seeding are documented in the *README.md* file located in the project root.

3. MoSCoW

The table below outlines the requirements set out in the brief. It describes the requirement and provides evidence of the web app fulfilling it.

ID	Requirement	Priority	Implementation Status & Evidence
A1	The system shall allow user to login using valid credentials and logout	Must Have	The system allows for unique users to login using their email and password.
A2	The system shall maintain state	Must Have	The system maintains state by using session storage.
A3	The system has role-based access control	Must Have	The system differentiates between the two types of users (Officers and Administrators) and directs them to their respective dashboards. Every action checks if the current logged in user has permissions to carry out said action.
B1	Create and manage classification of officers	Must Have	The system allows admins to create new officers and manage them.
B2	Assign classification officer to specific degree/programme	Must Have	The system allows admins to assign classification officers to programmes
B3	Create degree/programme with base classification rules	Should Have	Admins have the ability to create new programmes with base classification rules or they can edit those rules when creating the programme.
B4	Assign classification officer to more than one degree/programme	Should Have	With the database having a one-to-many relationship between programme and officer, officers are able to be assigned to more than one programme. This is also satisfied within the app UI.
C1	Classification Dashboard to show details of degree(s)/programme(s) and cohort summary.	Must Have	The officer dashboard displays the programmes they are currently assigned to and with a button view the roster of students in each programme in order to see a cohort summary and details of the programme.
C2	Cohort (candidate) data management i.e. CRUD	Must Have	Officers have the ability to create new students in a programme, view their records, update their marks accordingly, and delete their records according to CRUD methodology
C3	Automates classification rules on cohort and generates proposed classification	Must Have	In the student roster summary a button calculates the classification for each student and updates the page with proposed classification for each.
C4	Review and manage individual student records for manual overrides of system decision	Must Have	Each student record is able to be edited to override system decision or adjust marks given to the student for each module.
C5	Decision rationale log for each candidate.	Should Have	When an officer manually overrides a decision they have an input form to record why they have overridden the automated classification.
C6	Classification distribution summary	Should Have	The student roster page displays a bar chart that shows the distribution of classifications across the given programme.

ID	Requirement	Priority	Implementation Status & Evidence
C7	Export options for board of examiners	Could Have	In the student roster page, a button allows for a CSV to be exported with all relevant student data to be passed on to the board of examiners.
C8	Institute level dynamic classification rules	Could Have	While administrators set custom weighting scales for each programme, all resits are capped at a passing mark in the classification calculations.
C9	Defined per programme rather than institute level.	Could Have	Administrators can adjust the mark weighting accordingly when creating a new programme.
D1	Accuracy: Deterministic and consistent rule execution	Must Have	The system strictly enforces the 120-credit annual progression and the 40% resit cap as defined in the brief. By utilising programme-specific weighting variables from the data tier, the system ensures that award calculations are both mathematically accurate and reproducible across different degree pathways.
D2	Scalability: Supporting increase in degrees and users.	Must Have	The database holding data for students and programmes is scalable by its relational design. The modular code architecture allows for easy edits of the code should the need arise in the future. The API implemented allows for pulling data straight from the data tier without straining the main interface.
D3	Base Security measures to protect against bad actors	Must Have	Password hashing (using the <i>bcrypt</i> library) has been implemented into the app where the system compares the hash from the typed password against the stored hash. Each action that can be carried out by an admin or officer has numerous checks if they are authorised to carry out said actions based on their role, for example an administrator can't access the student roster let alone edit the marks of students. Each step of the application checks the user's role for authorisation of their actions.
D4	Design: modern and responsive user interface and experience	Must Have	Implementation of <i>EJS</i> code and <i>Bootstrap</i> collectively provides a clean and responsive UI and UX
D5	Versioning control using gitlab.eecs.qub.ac.uk account	Must Have	During development each step in the code was committed to the repository as evidenced in <i>gitlog.txt</i>
D6	Auditability: Full traceability of calculations and overrides	Could Have	Traceability is maintained through SQL logic that consistently handles resit caps and credit totals. Because the system ignores students with a manual override during automated calculation, there is a clear distinction between engine-generated results and human-logged rationale, ensuring a transparent audit trail for all awards.
E1	Modular Node/Express architecture with separated routes, controllers, and data layer	Should Have	Each user flow is divided into their own routes and controllers and has a separate data layer
E2	The system shall incorporate and utilise a stand-alone REST API to interface with the back-end data storage for relevant service operations using an	Should Have	A separate REST API on PORT 4000 provides database abstraction through JSON-formatted responses.

ID	Requirement	Priority	Implementation Status & Evidence
	appropriate set of REST API verbs and return data in a JSON format.		

4. System Architecture

4.1 Three-Tier Model

The system is divided into three distinct layers to prevent logic from leaking across the application. The presentation tier is built using *EJS* and *Bootstrap* to handle user interaction and display data to administrators and officers. The application (logic) tier contains the core business logic, including the classification engine and session management, implemented with *Node.js* and *Express.js*. The data tier utilises *MySQL*, a relational database that stores all persistent information, including student records, module marks, and programme-specific rules.

4.2 Modular MVC Implementation

The back-end follows the Model-View-Controller (MVC) pattern to separate routing logic from data processing. Files like *authRoutes.js* and *officerRoutes.js* act as traffic controllers, mapping URL endpoints to specific functions or pages. The *authController.js* and *officerController.js* files contain the functional logic; this is where the login and classification engines reside, respectively, processing raw database results into web interactions or academic outcomes. Database interactions are abstracted into a separate module using *mysql2/promise*, allowing for clean, asynchronous queries that do not block the application's main execution thread.

4.3 Stand-alone REST API

A stand-alone REST API was developed to run independently on **PORT 4000**. This service provides a database abstraction layer, returning student and programme data in JSON format. By separating the API from the main web application on **PORT 3000**, the system achieves better load distribution and remains open for future integration with other institutional tools.

5. Core Logic

5.1 Progression and Credit Validation

Before calculating a final average, the system validates that the student has met the institutional requirements for a Honours degree. Per the brief, the engine enforces strict rules before calculating any final awards. To be eligible for Honours, the system validates several conditions. The system checks if students accrued exactly 120 passed credits within each of the three

academic years, totalling the required 360 credits. The engine verifies that the sum of credits for modules with a mark greater or equal to 40 accrue the required maximum available credits. The system then checks if there are any outstanding fails in any modules the student had and the student did not pass a resit. If that is the case the system classifies that student as “Not eligible for Honours”. Year 1 is checked to ensure 120-credit pass requirement is met, but its marks do not contribute to the final classification average used to calculate the awards.

5.2 Grade Aggregation and Resit Capping

To calculate the yearly averages, the system applies the exact mathematical model defined in the brief. To maintain high performance and auditability, this formula is executed directly within the MySQL data tier using aggregation functions. A critical component of this logic is the automated enforcement of resit caps. The engine uses a CASE statement to ensure that any module passed through a resit contributes only a passing mark to the average. This aggregation is shown below.

```
SELECT
    s.student_id,
    SUM(CASE WHEN m.academic_year = 1 AND g.mark ≥ 40 THEN m.credits ELSE 0 END) AS
y1_credits,
    SUM(CASE WHEN m.academic_year = 2 AND g.mark ≥ 40 THEN m.credits ELSE 0 END) AS
y2_credits,
    SUM(CASE WHEN m.academic_year = 3 AND g.mark ≥ 40 THEN m.credits ELSE 0 END) AS
y3_credits,
    SUM(CASE WHEN g.mark < 40 THEN 1 ELSE 0 END) AS total_fails,
    ROUND(SUM(CASE WHEN m.academic_year = 2 THEN (CASE WHEN g.is_resit = 1 AND g.mark >
40 THEN 40 ELSE g.mark END) * m.credits ELSE 0 END) /
    NULLIF(SUM(CASE WHEN m.academic_year = 2 THEN m.credits ELSE 0 END), 0), 2) AS
l5_avg,
    ROUND(SUM(CASE WHEN m.academic_year = 3 THEN (CASE WHEN g.is_resit = 1 AND g.mark >
40 THEN 40 ELSE g.mark END) * m.credits ELSE 0 END) /
    NULLIF(SUM(CASE WHEN m.academic_year = 3 THEN m.credits ELSE 0 END), 0), 2) AS
l6_avg
FROM progr_students s
LEFT JOIN progr_grades g ON s.student_id = g.student_id
LEFT JOIN progr_modules m ON g.module_id = m.module_id
WHERE s.programme_id = ? AND s.manual_override = FALSE
GROUP BY s.student_id
```

5.3 Dynamic Weighting and Award Boundaries

Once eligibility is confirmed, the engine applies the weighting model to the Level 5 (Year 2) and Level 6 (Year 3) averages. While the system defaults to a 30/70 split, it dynamically pulls the *y2_weighting* and *y3_weighting* from the database to allow for programme-specific rules. The final classification is then derived from the weighted average:

$$\text{Final Average} = (L5 \text{ Avg} * Y2 \text{ Weight}) + (L6 \text{ Avg} * Y3 \text{ Weight})$$

The result is then mapped to the following academic boundaries:

Final Average	Classification Award
>= 70%	First Class Honours (1st)
60-69.99%	Upper Second Class (2:1)
50-59.99%	Lower Second Class (2:2)
40-49.99%	Third Class Honours (3rd)
< 40%	Fail

6. Security & Design

6.1 Base Security and Authentication

The HEdClass application implements robust security measures to protect sensitive student and institutional data from bad actors, fulfilling the base security requirements. Authentication for passwords is secured using the *bcrypt* library, which applies cryptographic hashing. This ensures that plain-text credentials are mathematically obscured and never stored directly in the database.

Beyond initial log in, the system relies on strict session management and Role-Based Access Control (RBAC) across the entire application tier. Server-side middleware verifies the user's role before processing any request. This strictly segregates access, ensuring that officers cannot access administrator endpoints, and unauthenticated traffic is immediately redirected away from the system.

6.2 Responsive UI and User Experience

To present a modern and responsive design, the presentation tier leverages the *Bootstrap CSS* framework alongside *Embedded JavaScript (EJS)* templating. The use of Bootstrap ensures the user interface is completely responsive, scaling cleanly from desktop administrative workstations to smaller mobile screens without loss of functionality. Furthermore, significant attention was given to the overall User Experience. The application utilises dynamic templating to adapt the interface based on the user's current session state. For example, redundant login prompts are hidden from authenticated users and replaced with direct dashboard navigation. To ensure seamless navigation, breadcrumbs are implemented across all application routes for both administrators and officers. In

addition to breadcrumbs, contextual back buttons are integrated onto specific sub-pages to provide clear, intuitive pathways for users moving through complex data workflows.

7. Conclusion

The HEdClass application successfully addresses the operational risks inherent in manual, spreadsheet-based degree classifications. By transitioning the calculation logic to a deterministic, automated engine, the system eliminates human mathematical error while ensuring strict compliance with institutional award regulations.

Through the rigorous application of the MoSCoW methodology, all core functional and non-functional requirements were met. The implementation of a robust three-tier architecture ensures the platform is scalable, maintainable, and secure.

Crucially the inclusion of features such as dynamic programme weighting, mandatory manual override rationales, and CSV exporting provides the board of examiners with a transparent and fully auditable classification process. Ultimately, HEdClass delivers a professional, comprehensive solution that improves the efficiency and accuracy of end-of-course academic administration.